

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_a6bedd49-683\agent-a62ae59627de5c2cf.jsonl`
- SHA-256: `192268151e5f473311c4339de91088049fc6fa4238565b36a56b26c70105d4f3`
- Source modified: `2026-07-06T23:57:05+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_a6bedd49-683`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T23:56:40.275Z)

Reviewing GitHub PR #55 "F6: persist policy decision traces" (branch feat/f6-policy-decisions -> main). ADR 0012 F6.
+387/-11, 7 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr55 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f6-policy-decisions:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F6 DOES:

- db.py: new policy_decisions table in BOTH _SCHEMA and @_migration(5)
_migration_005_policy_decisions (both CREATE TABLE IF NOT EXISTS).
Schema: (id PK, content_item_id FK content_items, policy_id FK policies, passed INT, trace_json TEXT, evaluated_at TEXT, UNIQUE(content_item_id, policy_id)).
Repos: record_policy_decision (UPSERT on (content_item_id,policy_id):
passed/trace_json/evaluated_at), get_policy_decision, list_policy_decisions_for_content.
_policy_trace_json = json.dumps(trace, sort_keys, compact). _row_to_policy_decision json.loads
+ validates list.
- models.py: PolicyDecision (frozen). policy.py: stage_results_to_trace(results) ->
[{stage_name,passed,reason,metadata}].
- telethon_client.py: _evaluate_policy_stages now returns (passed, results, policy_id).
_record_policy_decision(inserted, policy_id, policy_passed, policy_results):
guards 'if not policy_results or policy_id is None or inserted.id is None: return'; else
db.record_policy_decision(content_item_id=inserted.id, ...).
Called in on_new_message AND backfill AFTER insert_video (so the FK content_item exists).
on_new_message/backfill are '# pragma: no cover'.

VERIFIED by main reviewer: gate green (551 passed @ 94.34%); migration idempotent + reaches user_version=5 on a legacy v1 DB; a full pipeline trace serializes + persists; UPSERT is idempotent (1 row on re-record); policy decisions are stored SEPARATELY from delivery_attempts (addresses an earlier Track C finding).
Note deviations/limits already spotted: schema is one-row-per-decision-with-JSON-trace (ADR literally proposed per-stage rows); NO TTL/cleanup (ADR open question); metadata-rejected (Tier-1 FilterEngine) items get NO policy_decision row (only the evaluate_pipeline Tier 2-4 trace is recorded).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (migration data loss/corruption or a write bug that crashes ingestion/persists wrong data), medium (real latent bug/FK/serialization crash), low (note/deviation/growth).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong

behaviour. REFUTED if handled / can't occur / pre-existing / acceptable-deferred. PLAUSIBLE if unsettled. Corrected severity (migration data loss/corruption = high; latent serialization crash = medium; note = low). FINDING (persist_wiring): Metadata-rejected (Tier-1 FilterEngine) items persist no policy_decision row severity low | src/tg_video_filter/telethon_client.py:464 summary: When FilterEngine returns matched=False, policy stages are skipped so policy_results is empty and the guard `if not policy\_results ...: return` records nothing; these rejections leave no audit trace, so a future F7 /why cannot explain metadata-based rejections (only Tier 2-4 policy rejections are explainable). scenario: A video is rejected by a Tier-1 metadata filter (duration/size/resolution/content-type). matched=False, so `if matched and source is not None` (telethon_client.py:286) is skipped, policy_results stays []; _record_policy_decision's guard early-returns; no policy_decisions row exists. A later /why query for that item finds no persisted decision and cannot report why it was filtered. evidence: telethon_client.py:283-303 (policy stages only run when matched); telethon_client.py:472-473 guard `if not policy\_results or policy\_id is None or inserted.id is None: return`. Confirmed as a known deviation in the PR description; recording it as an explicit F7 explainability gap. rec: Deviation/growth note only -- acceptable for F6 scope. Flag for F7: metadata-rejection reasons live only in content_items/logs, not in policy_decisions, so /why must fall back to FilterEngine reasons for Tier-1 rejects.

2. assistant (2026-07-06T23:56:42.253Z)

I'll verify this finding by reading the cited code.

3. user (2026-07-06T23:56:43.581Z)

```

270             Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
271         )
272         if channel.id is not None:
273             source = await db.get_source_by_id(conn, channel.id)
274     except Exception:
275         log.warning(
276             "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
277         )
278
279     text = _message_text(message)
280     result = engine_holder[ENGINE_KEY].evaluate(meta)
281     matched = result.matched
282     reasons = list(result.reasons)
283     policy_passed = True
284     policy_results: list[StageResult] = []
285     policy_id: int | None = None
286     if matched and source is not None:
287         policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
288             conn,
289             user.id,
290             source,
291             _content_item_preview(
292                 user.id,
293                 source,
294                 message_id,
295                 document_id,
296                 meta,
297                 text,
298                 matched,
299             ),
300         )
301     if policy_results:
302         reasons.extend(result.reason for result in policy_results)
303     matched = policy_passed

```

```

304
305     record = VideoRecord(
306         user_id=user.id,
307         channel_id=channel_id,
308         message_id=message_id,
309         text=text,
310         document_id=document_id,
311         duration_s=meta.duration_s,
312         width=meta.width,
313         height=meta.height,
314         size_bytes=meta.size_bytes,
315         mime_type=meta.mime_type,
316         matched=matched,
317     )
318     try:
319         inserted = await db.insert_video(conn, record)
320     except sqlite3.IntegrityError:
321         # Lost a dedup race: a concurrent delivery of the same file
322         # (same document_id or same message instance) won the insert.
323         # The three-tier check above already classified this as new, so
324         # the collision means another task is processing (or has
325         # processed) it ? log and skip.
326         log.info(
327             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
328             user.id,
329             channel_id,
330             message_id,
331             document_id,
332         )
333         return
334     log.info(
335         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
336         user.id,
337         channel_id,
338         message_id,
339         matched,
340         reasons,
341     )
342     await _record_policy_decision(
343         conn,
344         inserted=inserted,
345         policy_id=policy_id,
346         policy_passed=policy_passed,
347         policy_results=policy_results,
348     )
349
350     if matched:
351         # Build link text for LINK/BOTH modes (None for FORWARD).
352         link_text = None
353         if delivery_mode != DeliveryMode.FORWARD:
354             channel_title = _channel_title(event)
355             link_text = build_link_text(
356                 message_link=getattr(message, "link", None),
357                 duration_s=meta.duration_s,
358                 height=meta.height,
359                 size_bytes=meta.size_bytes,
360                 channel_title=channel_title,
361             )
362         await deliver(
363             client,
364             message,
365             conn,
366             inserted.id,
367             delivery_mode,
368             link_text=link_text,

```

```

369         target=delivery_target,
370     )
371
372     return on_new_message
373
374
375 # --- helpers -----
376
377
378 def _extract_document_id(message: Message) -> int | None:
379     """Extract Telegram's ``document.id`` from a video message.
380
381     Returns None if the message has no document media or the id is absent.
382     This is the most reliable content-based dedup key: Telegram reuses the
383     same ``document.id`` when a file is forwarded across channels.
384     """
385     media = getattr(message, "media", None)
386     if isinstance(media, MessageMediaDocument):
387         media = media.document
388     if isinstance(media, Document):
389         return getattr(media, "id", None)
390     return None
391
392
393 def _message_text(message: object) -> str | None:
394     """Return caption/message text normalized for storage, or None if empty."""
395     text = getattr(message, "message", None)
396     if text is None:
397         text = getattr(message, "raw_text", None)
398     if not isinstance(text, str):
399         return None
400     text = text.strip()
401     return text or None
402
403
404 def _content_item_preview(
405     account_id: int,
406     source: Source,
407     message_id: int,
408     document_id: int | None,
409     meta: VideoMeta,
410     text: str | None,
411     matched: bool,
412 ) -> ContentItem:
413     """Build the ContentItem shape used by policy stages before insert."""
414     assert source.id is not None
415     return ContentItem(
416         account_id=account_id,
417         source_id=source.id,
418         provider=source.provider,
419         provider_message_id=str(message_id),
420         content_type="video",
421         text=text,
422         document_id=str(document_id) if document_id is not None else None,
423         duration_s=meta.duration_s,
424         width=meta.width,
425         height=meta.height,
426         size_bytes=meta.size_bytes,
427         mime_type=meta.mime_type,
428         dedup_key=db.compute_dedup_key(
429             provider=source.provider,
430             document_id=str(document_id) if document_id is not None else None,
431             source_id=source.id,
432             provider_message_id=str(message_id),
433             size_bytes=meta.size_bytes,

```

```

434         duration_s=meta.duration_s,
435         height=meta.height,
436         width=meta.width,
437     ),
438     matched=matched,
439 )
440
441
442 async def _evaluate_policy_stages(
443     conn: aiosqlite.Connection,
444     account_id: int,
445     source: Source,
446     content_item: ContentItem,
447 ) -> tuple[bool, list[StageResult], int | None]:
448     """Evaluate ADR 0012 policy stages for a candidate item."""
449     policy = await db.get_default_policy(conn, account_id)
450     if policy is None:
451         return True, [], None
452     # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453     # user-facing enable command must land with fetcher/budget wiring.
454     passed, results = await evaluate_pipeline(
455         PolicyContext(content_item=content_item, source=source, policy=policy),
456         [
457             TopicPreferenceStage(),
458             UrlReputationStage(),
459             CaptionSafetyStage(),
460             DeepInspectionStage(),
461         ],
462     )
463     return passed, results, policy.id
464
465
466 async def _record_policy_decision(
467     conn: aiosqlite.Connection,
468     *,
469     inserted: VideoRecord,
470     policy_id: int | None,
471     policy_passed: bool,
472     policy_results: list[StageResult],
473 ) -> None:
474     """Persist a policy trace after the content item has been inserted."""
475     if not policy_results or policy_id is None or inserted.id is None:
476         return
477     await db.record_policy_decision(
478         conn,
479         content_item_id=inserted.id,
480         policy_id=policy_id,
481         passed=policy_passed,
482         trace=stage_results_to_trace(policy_results),
483     )
484
485
486 def _peer_id(peer: object) -> int:
487     """Best-effort numeric id for the peer of a message.
488
489     Telegram represents channel/group/user peers as different TL types; we

```

4. assistant (2026-07-06T23:56:46.945Z)

The finding is a low-severity deviation note. Let me verify the two key claims: (1) policy stages are skipped when `matched=False`, and (2) the guard early-returns when `policy\_results` is empty. Both are visible in the code I just read. Let me confirm the FilterEngine behavior and that this is confined to this PR's diff.

5. user (2026-07-06T23:56:48.995Z)

```

diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index c5f760f..37089ae 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -52,6 +52,7 @@ from tg_video_filter.policy import (
    TopicPreferenceStage,
    UrlReputationStage,
    evaluate_pipeline,
+   stage_results_to_trace,
)

log = logging.getLogger(__name__)
@@ -279,8 +280,11 @@ def make_new_message_handler(
    result = engine_holder[ENGINE_KEY].evaluate(meta)
    matched = result.matched
    reasons = list(result.reasons)
+   policy_passed = True
+   policy_results: list[StageResult] = []
+   policy_id: int | None = None
    if matched and source is not None:
-       policy_passed, policy_results = await _evaluate_policy_stages(
+       policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
            conn,
            user.id,
            source,
@@ -335,6 +339,13 @@ def make_new_message_handler(
    matched,
    reasons,
)
+   await _record_policy_decision(
+       conn,
+       inserted=inserted,
+       policy_id=policy_id,
+       policy_passed=policy_passed,
+       policy_results=policy_results,
+   )

    if matched:
        # Build link text for LINK/BOTH modes (None for FORWARD).
@@ -433,14 +444,14 @@ async def _evaluate_policy_stages(
    account_id: int,
    source: Source,
    content_item: ContentItem,
-) -> tuple[bool, list[StageResult]]:
+) -> tuple[bool, list[StageResult], int | None]:
    """Evaluate ADR 0012 policy stages for a candidate item."""
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
-       return True, []
+       return True, [], None
    # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
    # user-facing enable command must land with fetcher/budget wiring.
-   return await evaluate_pipeline(
+   passed, results = await evaluate_pipeline(
        PolicyContext(content_item=content_item, source=source, policy=policy),
        [
            TopicPreferenceStage(),
@@ -449,6 +460,27 @@ async def _evaluate_policy_stages(
        DeepInspectionStage(),
    ],
)
+   return passed, results, policy.id
+
+
+async def _record_policy_decision(

```

```

+     conn: aiosqlite.Connection,
+     *,
+     inserted: VideoRecord,
+     policy_id: int | None,
+     policy_passed: bool,
+     policy_results: list[StageResult],
+) -> None:
+     """Persist a policy trace after the content item has been inserted."""
+     if not policy_results or policy_id is None or inserted.id is None:
+         return
+     await db.record_policy_decision(
+         conn,
+         content_item_id=inserted.id,
+         policy_id=policy_id,
+         passed=policy_passed,
+         trace=stage_results_to_trace(policy_results),
+     )

def _peer_id(peer: object) -> int:
@@ -485,5 +517,6 @@ __all__ = [
    "make_new_message_handler",
    "_content_item_preview",
    "_evaluate_policy_stages",
+   "_record_policy_decision",
    "_message_text",
]

```

6. assistant (2026-07-06T23:56:57.992Z)

The finding is confirmed as a factual, accurate description of the code behavior. The code at lines 286, 301, 475-476 clearly shows:

1. Policy stages only run when `matched` and source is not None` (line 286)
2. When `matched=False` (Tier-1 metadata rejection), `policy\_results` stays `[]`
3. The guard `if not policy\_results or policy\_id is None or inserted.id is None: return` (line 475) early-returns, so no `policy\_decisions` row is written

The description is accurate. This is a design deviation/observability gap, not a bug: no crash, no data corruption, no wrong data persisted. The item is still recorded in `content\_items` with `matched=False` and reasons are logged (line 334-341). The finding itself explicitly frames it as an "acceptable-deferred" F7 growth note, and the PR description already discloses it as a known deviation.

Per the severity rubric (note/deviation/growth = low), this is correctly rated low. It genuinely ships in this PR's diff (the `\_record\_policy\_decision` wiring is new), and the described behavior is real. It is REFUTED as a *defect* in the "bug" sense but the finding does not claim it is a bug ? it explicitly classifies it as a deviation/growth note. So the correct verdict is CONFIRMED at low severity: the described behavior is real, ships in this diff, and is accurately characterized as a deferred observability gap rather than a fault.

7. user (2026-07-06T23:57:04.984Z)

Structured output provided successfully